



ARA0110 PADRÕES DE PROJETO

Aula 09: Padrões de projeto comportamentais - parte 4





RECAPITULANDO





PADRÕES COMPORTAMENTAIS

Os padrões de projeto comportamentais focam em algoritmos e atribuição de responsabilidades entre objetos, descrevendo padrões de comunicação entre eles.

Nesses padrões, encontramos uso intensivo de polimorfismo.

TEMPLATE METHOD





sai maluco
todo dia isso

DEFINIÇÃO

Definir o esqueleto de um algoritmo em uma operação, postergando alguns passos para as subclasses. Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar a estrutura do mesmo.

GoF, Padrões de Projetos. p.301

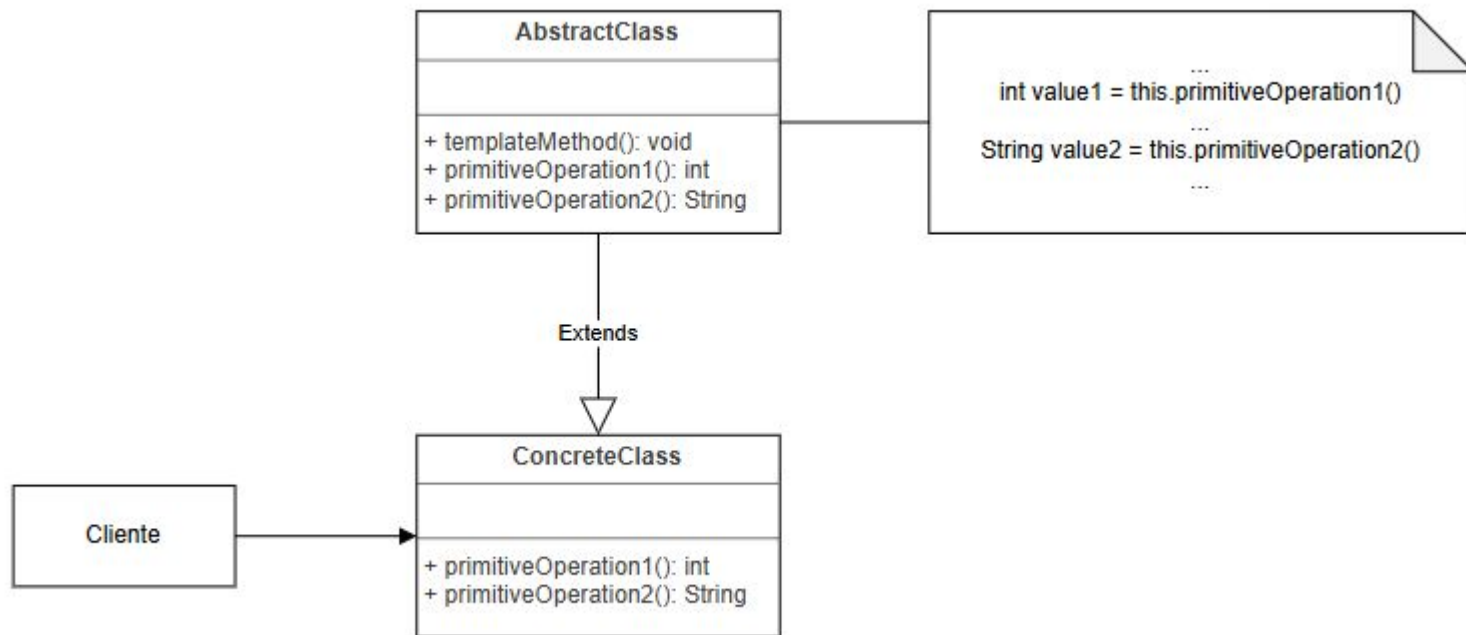


sai maluco
todo dia isso

MOTIVAÇÃO

A ideia é evitarmos duplicação de código com algoritmos que compartilham uma lógica comum, mas que possuem variações em passos específicos. Por exemplo, a ação de almoçar compartilha passos em comum (pegar um prato, servir, sentar, comer, lavar a louça suja) mas que pode sofrer variações em alguns passos (cozinhar ou pedir delivery).

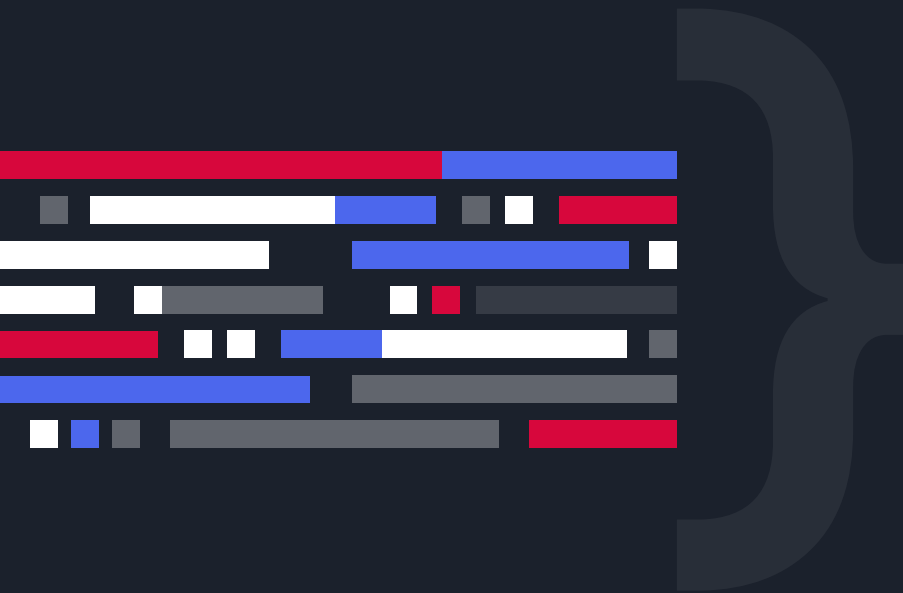
DIAGRAMA





PARTICIPANTES

- **AbstractClass:** classe abstrata que define os passos de um algoritmo e declara operações abstratas que variam no fluxo
- **ConcreteClass:** implementação das operações que variam no fluxo, para cada caso particular
- **Cliente:** elemento que utiliza o algoritmo a partir das classes concretas das operações



UTILIZAÇÃO

Cada subclasse implementa os trechos do algoritmo que variam para cada cenário. O cliente utiliza a operação completa a partir das subclasses que implementam a abstração (uma vez que não é possível utilizar o algoritmo a partir da abstração somente).

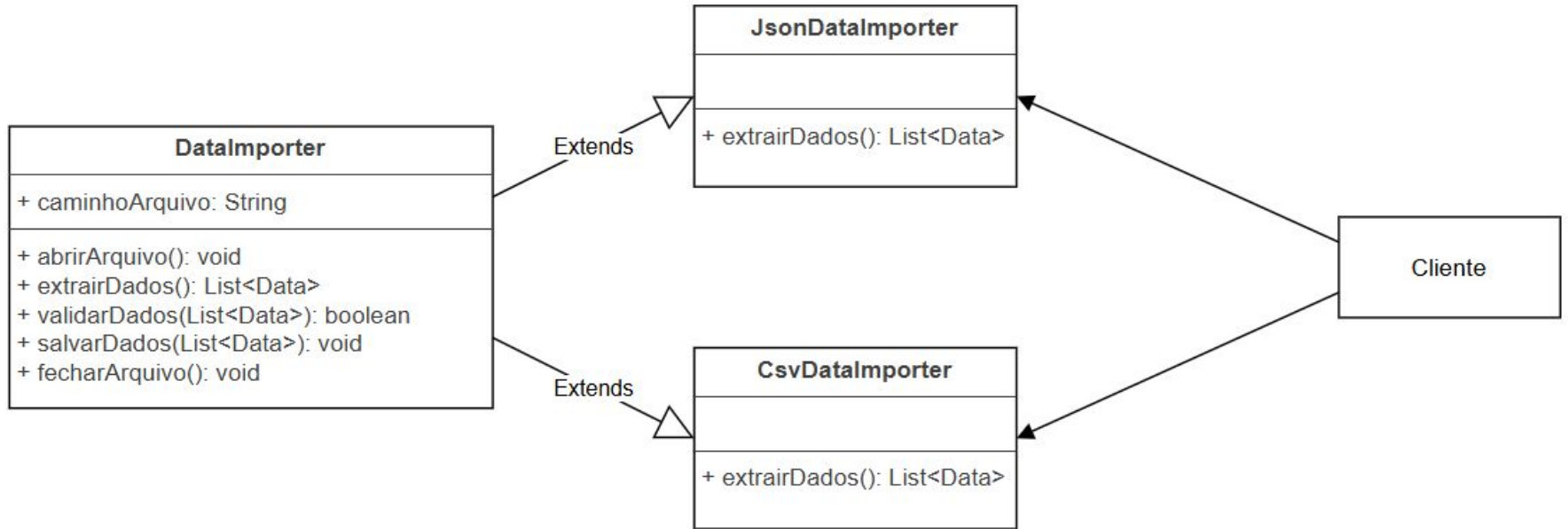


HORA DE EXERCÍCIO

Um dashboard de visualização pode processar arquivos de diferentes formatos (CSV, JSON) para alimentar uma base de visualização de métricas. O processo de importação dos dados segue o mesmo fluxo independente do formato: abrir o arquivo, extrair os dados, validar, salvar no banco e fechar o arquivo.

Vamos implementar esse sistema.

DIAGRAMA





HORA DE EXERCÍCIO

Em um jogo, diferentes tipos de inimigos (Orc, Mago) podem ter comportamentos baseados na mesma lógica de decisão: detectar o inimigo, avaliar distância, executar ação e retornar ao estado base. Um Orc pode correr até o inimigo e atacá-lo fisicamente; Um mago mantém distância do inimigo e lança magias para atingi-lo.

DIAGRAMA?



APLICABILIDADE

- Implementar partes invariantes de um algoritmos só uma vez, e delegar às subclasses a implementação do comportamento variante
- Quando se quer controlar extensões de subclasses, com o intuito de evitar duplicação de código



CONSEQUÊNCIAS

- Controle na duplicação de partes comuns de um algoritmo
- Subclasses implementam somente o necessário
- Atenção na chamada dos métodos herdados, evitando esquecê-los

VISITOR





DEFINIÇÃO

Representar uma operação a ser executada nos elementos de uma estrutura de objetos. Visitor permite definir uma nova operação sem mudar as classes dos elementos sobre os quais opera.

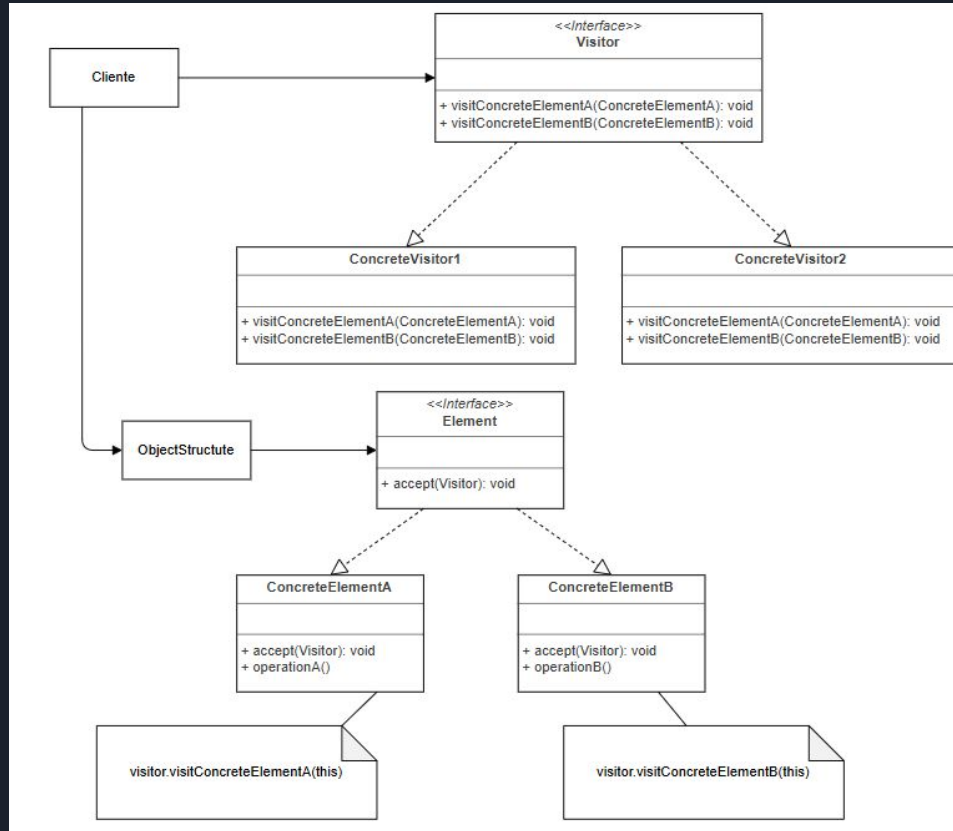
GoF, Padrões de Projetos. p.305



MOTIVAÇÃO

Em um sistema de relatórios, temos os dados visualizados podendo ser exportados em diferentes formatos (PDF, XLSX, CSV etc). Com um visitor, conseguimos adicionar novas operações a esse processo de exportação sem modificar as classes existentes, impedindo a violação do Princípio Aberto/Fechado.

DIAGRAMA





PARTICIPANTES

- **Visitor:** interface que declara uma operação *visit* para cada classe ConcreteElement na estrutura do objeto
- **ConcreteVisitorX:** implementação da operação *visit* para cada classe
- **Element:** interface que declara uma operação *accept* que recebe um objeto Visitor
- **ConcreteElementX:** implementação da operação *accept* para receber um Visitor



PARTICIPANTES

- **ObjectStructure:** estrutura de objetos que enumera cada elemento presente no programa
- **Cliente:** elemento que acessa os objetos Element registrados e repassa os objetos Visitor para cada um



UTILIZAÇÃO

Listamos todos os objetos de elementos da aplicação em uma estrutura que pode ser acessada na sua execução. Ao acessar um elemento, o cliente repassa um objeto Visitor para a instância, que irá executar a operação de visita de acordo com a implementação de cada método no visitante.

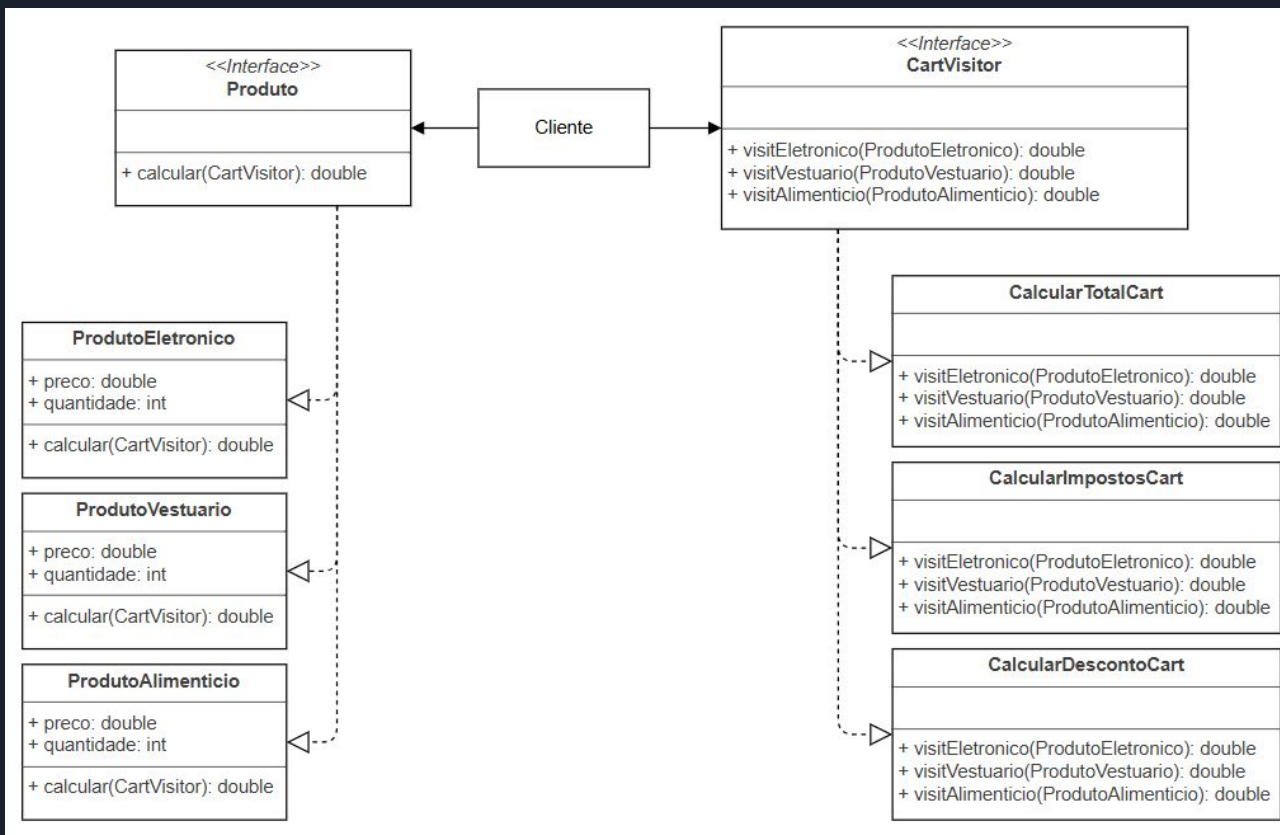


HORA DE EXERCÍCIO

Em um e-commerce, o carrinho pode ter produtos de vários tipos (eletrônicos, vestuário, alimentos) em uma compra. Na finalização da compra, precisamos calcular o total, aplicar descontos específicos e calcular impostos. O imposto para um produto eletrônico possui uma dinâmica diferente em relação a um produto alimentício.

Vamos implementar essa operação.

DIAGRAMA



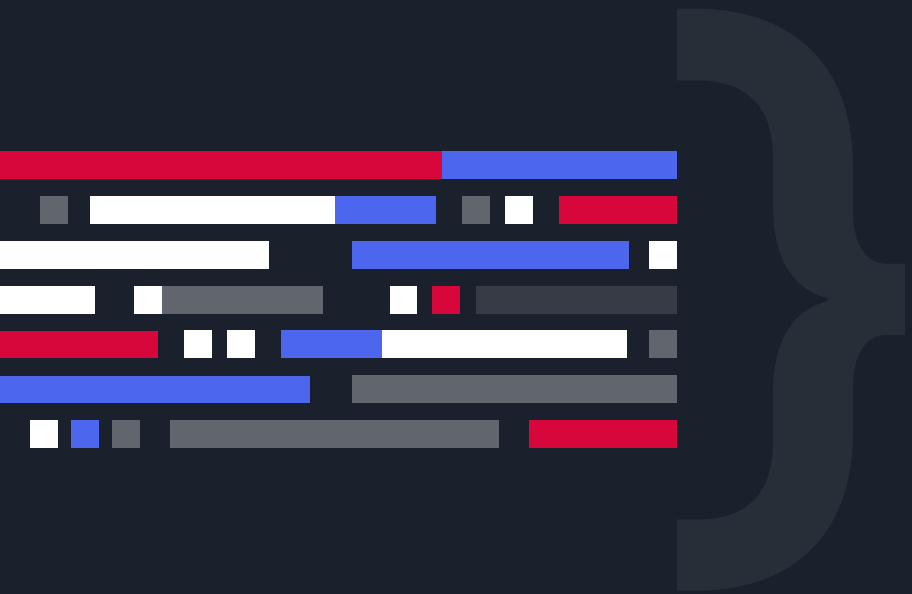


HORA DE EXERCÍCIO

Em uma estrutura hierárquica de arquivos e pastas, você precisa realizar operações como calcular o tamanho total ocupado, encontrar arquivos duplicados ou alterar permissões.

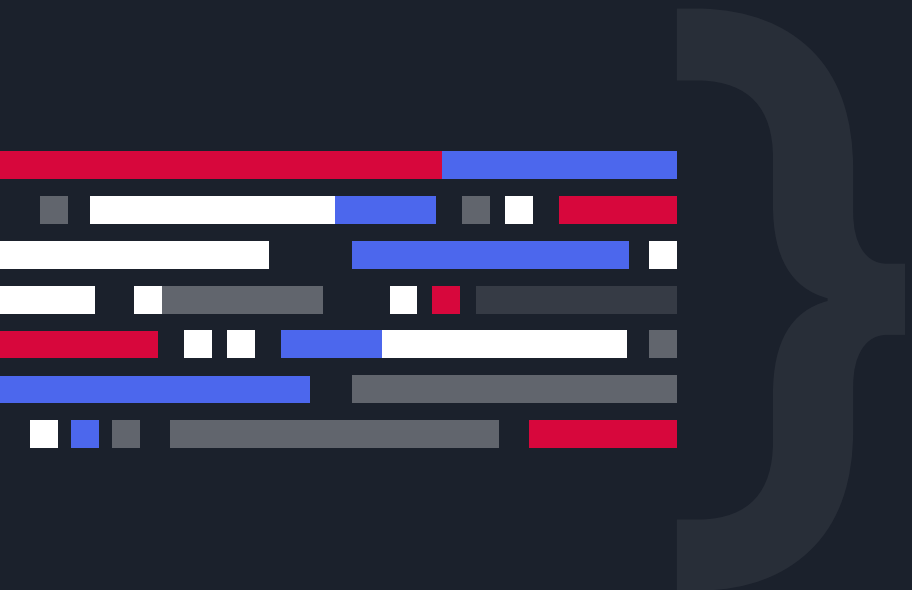
Implemente uma solução para executar essas operações sobre a hierarquia do disco.

DIAGRAMA?



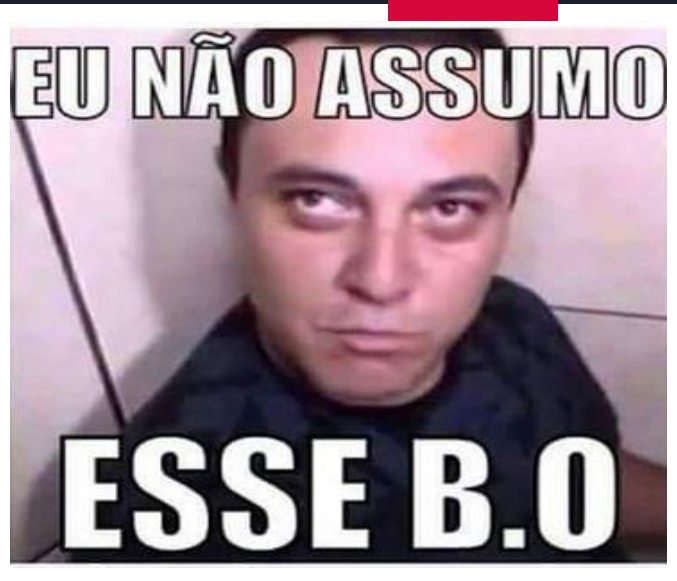
APLICABILIDADE

- Manter uma estrutura de objetos que possuem muitas classes de objetos com interfaces divergentes que precisam executar operações
- As classes da estrutura de objetos raramente mudam, enquanto as operações sobre a estrutura variam bastante



CONSEQUÊNCIAS

- Tornar fácil a adição de novas operações
- Um visitante reúne operações relacionadas e separa as não relacionadas
- É difícil acrescentar novas classes ConcreteElement



PADRÕES COMPORTAMENTAIS

Ao longo dos exemplos, vimos que as estruturas de objetos podem ser bastante complexas de manter sem um planejamento prévio. Usar os padrões comportamentais nos dão organização e flexibilidade nas operações realizadas pelas aplicações.

Combinadas a outros padrões, enriquecemos a aplicação com uma boa estrutura de componentes.



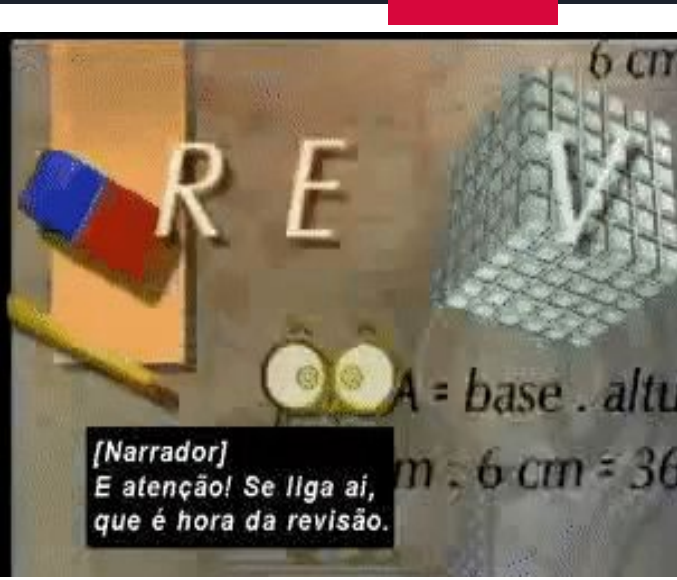
PADRÕES

- **Chain of Responsibility:** encadear objetos para atender uma solicitação quando for possível
- **Command:** parametrizar clientes com diferentes solicitações
- **Interpreter:** definir um interpretador de sentenças em uma linguagem padrão
- **Iterator:** prover funcionalidades para acesso sequencial, sem expor o estado interno
- **Mediator:** definir um objeto que encapsula a forma como um conjunto de objetos interage
- **Memento:** externalizar o estado interno de um objeto, sem ferir o encapsulamento



PADRÕES

- **Observer:** atualizar objetos interessados na mudança do estado interno de um outro objeto alvo
- **State:** alterar o comportamento de um objeto quando o seu estado interno mudar
- **Strategy:** variar algoritmos de uma família, encapsulando-os independentemente dos clientes que o utilizam
- **Template Method:** definir o esqueleto de um algoritmo, concentrando partes comuns e abstraindo passos variantes
- **Visitor:** definir operações sem modificar os elementos sobre os quais executam



RECAPITULANDO

- Padrões de projeto comportamentais
- Padrão Template Method
- Padrão Visitor
- Revisão dos padrões de projeto comportamentais

OBRIGADO !



"TOMA AQUI, Ó! TOMA AQUI!"
Galerito vs Gil da Esfirra